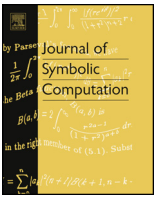
Journal of Symbolic Computation 128 (2025) 102375

Contents lists available at ScienceDirect

Journal of Symbolic Computation

journal homepage: www.elsevier.com/locate/jsc



Invariant neural architecture for learning term synthesis in instantiation proving

Jelle Piepenbrock [a,b], Josef Urban [b], Konstantin Korovin [c], Miroslav Olšák [d], Tom Heskes [a], Mikoláš Janota [b]

[a] Radboud University, Nijmegen, the Netherlands
[b] Czech Technical University, Prague, Czech Republic
[c] The University of Manchester, Manchester, United Kingdom
[d] University of Cambridge, Cambridge, United Kingdom

ARTICLE INFO

Article history:
Received 18 September 2023
Received in revised form 14 August 2024
Accepted 23 August 2024
Available online 28 August 2024

Keywords:
Automated theorem proving
Graph neural networks
Instantiation

ABSTRACT

The development of strong CDCL-based propositional (SAT) solvers has greatly advanced several areas of automated reasoning (AR). One of the directions in AR is therefore to make use of SAT solvers in expressive formalisms such as first-order logic, for which large corpora of general mathematical problems exist today. This is possible due to Herbrand's theorem, which allows reduction of first-order problems to propositional problems by instantiation. The core challenge is synthesizing the appropriate instances from the typically infinite Herbrand universe.

In this work, we develop a machine learning system targeting this task, addressing its combinatorial and invariance properties. In particular, we develop a GNN2RNN architecture based on a graph neural network (GNN) that learns from problems and their solutions independently of many symmetries and symbol names (addressing the abundance of Skolems), combined with a recurrent neural network (RNN) that proposes for each clause its instantiations. The architecture is then combined with an efficient ground solver and, starting with zero knowledge, iteratively trained on a large corpus of mathematical problems. We show that the system is capable of solving many problems by such educated guessing, finding proofs for 32.12% of the training set. The final trained system solves 19.74% of the unseen test data on its own.

E-mail address: jelle.piepenbrock@ru.nl (J. Piepenbrock).

https://doi.org/10.1016/j.jsc.2024.102375
0747-7171/© 2024 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

We also observe that the trained system finds solutions that the iProver and CVC5 systems did not find.

© 2024 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

1. Introduction

Quantifiers lie at the heart of mathematical logic, modern mathematics and reasoning. They enable expressing statements about infinite domains. Practically all today's systems used for formalization of mathematics and software verification are based on expressive foundations such as first-order and higher-order logic, set theory and type theory, that make essential use of quantification.

Instantiation is a powerful tool for formal reasoning with quantifiers. The power of instantiation is formalized by *Herbrand's theorem* (Herbrand, 1930), which states that a set S of first-order clauses is unsatisfiable if and only if there is a finite set of ground instances of S that is (propositionally) unsatisfiable. Herbrand's theorem further states that it is sufficient to consider instantiations from the *Herbrand universe*, which consists of terms with no variables (*ground terms*) constructed from the symbols appearing in the problem. This fundamental result has been explored in automated reasoning (AR) systems since the 1950s (Davis, 2001; Davis and Putnam, 1960; Schulz, 2002). In particular, once the right instantiations are discovered, the problem typically becomes easy to decide by state-of-the-art SAT solvers (Silva et al., 2009; Korovin, 2013).

Coming up with the right instantiations is often difficult. As soon as there are non-nullary functions (e.g., the successor, $s(x)$) in a problem, its set of ground terms (the Herbrand universe) becomes infinite (e.g., $s(0), s(s(0)), \ldots$). There are typically many non-nullary functions in common mathematical problems. The general undecidability of theorem proving is obviously connected to the hardness of finding the right instantiations, which includes finding arbitrarily complex mathematical objects.

Investigating the interaction between symbolic computation, in the form of an automated theorem proving system, and machine learning, to learn heuristics to guide the choices of the provers, is a promising direction. In this way, both the powerful exact reasoning of the prover and the approximate, heuristic reasoning of machine learning can be exploited.

Contributions: In this work we develop a completely naming-agnostic machine learning (ML) method that automatically proposes suitable instantiations. The system learns to instantiate from scratch, improving in an iterative fashion. This is motivated both by the growing ability of ML methods to prune the search space of automated theorem provers (ATPs) (Kaliszyk et al., 2018), and also by their growing ability to synthesize various logical data (Gauthier, 2020; Urban and Jakubův, 2020). In particular:

1. We decompose the problem of theorem proving into a neural instantiation step followed by a ground solver (Section 2.2). We devise an incremental procedure by which a predictor can propose instantiations (Section 2.4).
2. We develop a targeted GNN2RNN neural architecture based on a graph neural network (GNN) that learns to represent the problems and their clauses independently of many symmetries and symbol names (addressing the abundance of Skolems), combined with a recurrent neural network (RNN) that proposes for each clause its instantiations based on the GNN characterization (Section 2.5).
3. We construct an initial corpus of instantiations by repeatedly running a randomized grounding procedure followed by a ground solver (Section 2.2) on 113 332 clausal ATP problems extracted from the Mizar Mathematical Library. We analyze the solutions, showing that almost two-thirds of the instances contain newly introduced Skolem symbols created by the clausification (Section 2.8).
4. The GNN2RNN is trained and used to propose instances for the problems. Its training starts with the randomized solutions and continues by incrementally learning from its own successful predictions. We show that the system can predict the appropriate instances and that it can continually expand the set of proven problems, improving its performance (Section 3).

5. The trained neural network when combined with the ground solver is shown to be able to solve 19.74% of testing problems by making educated guesses (Section 3). The system finds solutions that were not found by existing systems such as iProver and CVC5. To our knowledge, this is the first naming-agnostic neural system for general synthesis of relevant elements from arbitrary Herbrand universes.

2. Methods

2.1. Solving by instantiation

First, we give an overview of the problem and recall existing approaches, after which we explain our specific methods and solutions. For our examples and most of our treatment in this work, we will work in first-order logic, but instantiation can be used in a broader context, notably in higher-order and SMT solving (Clarke et al., 2018). In particular, we will work with clausal first-order problems: the problems are expressed as a conjunction of *clauses*. These clauses contain *literals* which are connected by disjunctions. The literals have as their head symbol a *relational symbol*, which denotes a predicate that assigns a truth value, or a negation, which negates that truth value. Under these symbols there are *terms* which can be constructed from a combination of *variables*, *function symbols* and *constants*. A term without variables is called a *ground* term.

The basic setting is that we have a set of (universally quantified) clauses that can be instantiated to a set of clauses without variables (*ground* clauses) which is unsatisfiable. Consider the following set of clauses:

$$\forall x. P(f(x)) : \text{Axiom 1} \tag{1}$$

$$\forall x. \neg P(x) \vee Q(x) : \text{Axiom 2} \tag{2}$$

$$\neg Q(f(g(c))) : \text{Negated Conjecture} \tag{3}$$

Axioms 1 and 2 can be instantiated to create the ground clauses $P(f(g(c)))$ and $\neg P(f(g(c))) \vee Q(f(g(c)))$ causing a propositional contradiction with the clause $\neg Q(f(g(c)))$. In this example, the substitutions $x \to g(c)$ and $x \to f(g(c))$ are not hard, but in general, the process of choosing the right instantiations of even a single quantified clause can be non-trivial. There can be many variables in one clause, and, in the general case, we have to choose the right instantiations for many clauses at once to arrive at a contradiction (and we might even need several different instantiations of the *same* clause). This is a problem of high combinatorial complexity, and there are several existing strategies to choose the right terms for the corresponding variables.

The automated theorem prover iProver is based on the *Inst-Gen calculus* (Korovin, 2013), in which the first-order instantiation and grounding are interwoven with calls to a propositional SAT solver. The propositional model generated by the SAT solver is then used to decide how to instantiate. In SMT (Clarke et al., 2018), there are several methods in use for instantiation, for example *enumerative instantiation*, which simply enumerates term substitutions, prioritizing the ground terms created earlier, *E-matching* (De Moura and Bjørner, 2007), in which a pattern-matching strategy is used to decide instantiations, and *model-based instantiation* (Ge and de Moura, 2009).

In Fig. 1A, we show the general framework for these solvers. There is a solver, that can generate instantiations, which are passed to a ground solver (for example, a propositional SAT solver in combination with congruence closure procedures). This ground solver can produce SAT or UNSAT outcomes. In the case of SAT, the generated propositional model can be used to decide the next instantiations. However, it is difficult to decide what instantiations to use. In the next sections, we explain the components of our learning-based approach to this task.

2.2. Combining an instantiator with a ground solver

Our general procedure is shown schematically in Fig. 1B. We train a suitable neural architecture (Section 2.5) to do incremental instantiation (Section 2.4) which is combined with an efficient ground solver.

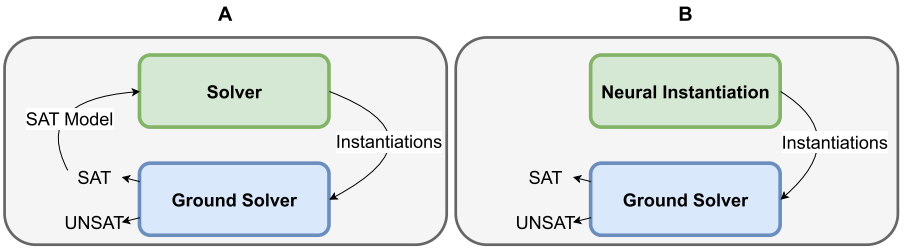


Fig. 1. Solving problems by instantiation. On the left, in subfigure A, we show a general scheme, which fits certain modes for existing automated theorem provers such as iProver and SMT solvers such as CVC5. The first-order prover or SMT solver can generate instantiations and call the ground solver. The ground solver either finds a contradiction (and thus a proof) and returns UNSAT, or it finds a suitable interpretation of the problem and returns SAT with a propositional model, which the solver can use. On the right, in subfigure B, we show our approach, in which the instantiation is done solely by a neural component. Note that we currently do not use the SAT models generated, but are using a "single-shot" approach.

There are several ways how to combine instantiation of clausal first-order problems with decidable and efficient (un)satisfiability checking of their proposed ground instances. The most direct approach (used in instantiation-based ATPs such as iProver (Korovin, 2008)) is to explicitly add axioms for equality, allowing their instantiation as for any other axioms, and directly use SAT solvers for the ground checking. An alternative approach is to avoid explicit addition of the equality axioms, and instead use combinations of SAT solvers with ground congruence closure (Detlefs et al., 2005; Nieuwenhuis and Oliveras, 2007).

We have explored both approaches and ultimately decided to use the latter in this work. The main reason is that the combinations of SAT solvers with ground congruence closure are today very efficiently implemented (Barrett et al., 2021), posing practically no issues even with thousands of instances. Using the most direct approach would, on the other hand, require a large number of additional instances of the equality axioms to successfully solve the ground problems. In our preliminary measurements, the average ratio of such necessary additional instances was over 40%, which would exponentially decrease the chance of predicting the right set of instances.

2.3. Ground solver for CC + SAT

We use as our ground solver an efficient combination of a SAT solver with ground congruence closure (CC + SAT). In CC + SAT, the SAT solver abstracts the ground atoms as propositional variables and starts producing satisfying assignments (models) of this abstraction, which are then checked against the properties of equality (reflexivity, symmetry, transitivity, congruence). This process terminates when a model is found that satisfies the equality properties, or when the SAT solver runs out of models to try. In that case, the original ground problem is unsatisfiable.

2.4. Incremental instantiation procedure

We first give a high-level view of how the neural network predicts the instantiations. The neural prediction is decomposed into levels, where each level deepens the non-ground terms. The idea is depicted in Fig. 2. At each level, the network predicts a single function symbol for each variable of a given clause C. Then, a new instance C_1 of C is created by replacing the variables with the proposed function symbols and fresh variables as their arguments. This whole process is iterated.

Besides the increasing depth, the network also needs to be able to deal with an arbitrary number of variables in each clause. This is handled in an RNN fashion—variables are being predicted in a fixed order and the information about the previous predictions is stored in a hidden state. Note that the number of symbol predictions needed increases or decreases depending on the arity of the predicted symbols. In particular, if all predicted symbols are constants (symbols of arity 0), no new variables are generated and the process stops. We are most often required to predict instances for multiple clauses at the same time, as well as in some cases multiple instances for the same clause.

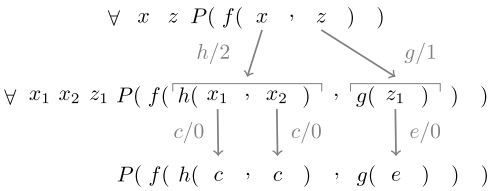


Fig. 2. Term instantiation through incremental deepening. In the figure, there are two instantiation steps, one after the other.

As an example, this iterative process would be able to generate the term $f(c,(f(c,c))$ starting from $f(x,y)$, but it would take 2 levels: one in which x is set to c and y is set to f, and another level in which two fresh variables generated under f are both instantiated with c.

2.5. Neural network architecture

Mathematical problems often have many symmetries, making them challenging for the naive use of off-the-shelf sequence-based learning methods. Clausal problems are invariant under the reordering of clauses and literals, renaming of variables in each clause, and also under consistent renaming of symbols in a problem. To address this, we base our architecture on a graph neural network (GNN) with such properties proposed by (Olšák et al., 2020) and used so far in several ATP tasks (Jakubův et al., 2020; Chvalovský et al., 2021) to *classify existing objects*. In this work, we re-implement that GNN in PyTorch (Paszke et al., 2019), and add to it a novel anonymous, signature-bound recurrent neural network (RNN) that allows us to also *generate new objects* (clause instantiations). The relation between the graph neural network and the RNN is illustrated by Fig. 3. To our knowledge, this is the first ML architecture combining such strong invariant and nameless problem encoding with the need for non-anonymous symbolic decoding (synthesis).

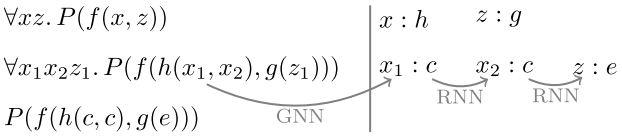


Fig. 3. Schematic of RNN Predictions corresponding to Fig. 2. A GNN communicates via variable representations to the RNN predictor and previous predictions within the same clause are communicated by an RNN hidden state. Fig. 5 shows a more detailed procedure.

Next, we detail the neural network architecture. Code and data to run the system is available on Github at https://github.com/JellePiepenbrock/neural-synthesis.

2.5.1. Graph neural network

The GNN architecture we use is specifically constructed to be invariant to symbol names, as it treats the input clauses in a fully anonymous manner (Olšák et al., 2020). In addition, the network has a notion of negation. The particular structure of clausified first-order logic problems is taken into account, with clause nodes able to communicate with literals, terms able to communicate with their sub-terms, and all symbols able to directly communicate with all terms they are in. The graph neural network is invariant to clause order permutations and literal permutations.

In Fig. 4, we show a schematic graph corresponding to the example in Section 2.1 that indicates how the nodes are connected, from the graph neural network's perspective. For reading convenience, we repeat here the three clauses from that example: Axiom 1: $\forall x.P(f(x))$, Axiom 2: $\forall x.\neg P(x) \vee Q(x)$ and the negated conjecture $\neg Q(f(g(c)))$.

There are *term nodes* **T** (three types of these are distinguished in the representation: 'standard' terms, variables and literal), the *symbol nodes* **S** (which can be function symbols or relational) and the *clause nodes* **C** (which can be axioms or conjecture clauses).

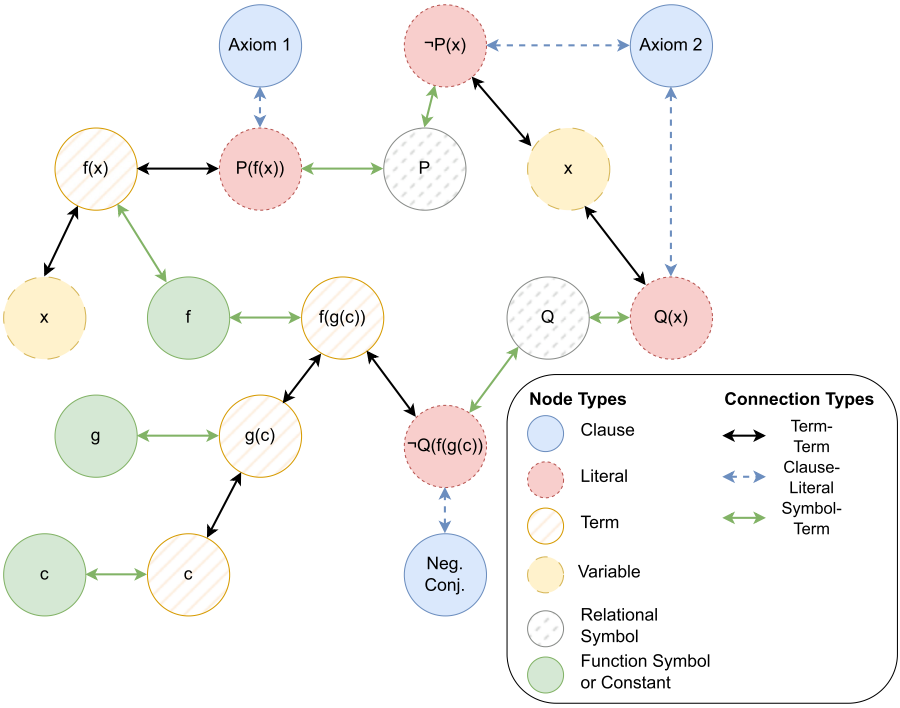


Fig. 4. A schematic representation of the graph corresponding to the example in Section 2.1. Note that several aspects of the GNN's message passing structure have been suppressed for visual clarity, such as the treatment of polarity and the handling of argument ordering in terms with function arities higher than 1. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

When a problem is processed by the graph neural network, each node in the graph is first given an initial numerical vector representation according to its type. The representations of these nodes are then updated according to the representations of each node's neighbors (a process called *message passing*). Each node aggregates the incoming vectors by aggregation operators, in our case the element-wise *mean* and *max* functions. After aggregation, a linear transformation in the form of a matrix with learnable parameters is applied to the aggregated representation. Then a non-linear *activation* function is applied which creates an updated vector representation. This update procedure can be iterated, so that nodes that are not direct neighbors can also update according to each other's representations.

Different matrices are used for the transformation in different connection types (for example, the transformation that updates clause representation has different parameters than the transformation that updates symbol representations.) During the training procedure, these parameters will be tuned to produce node vector representations that are useful to predict the instantiation of clauses.

For a full exposition of the update equations used, we refer to Section 2 of (Olšák et al., 2020). For our purposes here, it is enough that after several message passing rounds, the GNN outputs updated vector representations for the nodes in the graph. These vector representations of the nodes, that can incorporate structural information about the graph surrounding each node, are then used to predict how to instantiate the clauses.

2.5.2. RNN function symbol prediction

We modify the original GNN architecture to allow the network to produce instantiations for each clause by using an RNN after running the GNN (Fig. 5). The setup is as follows for the first variable in each clause. We predict an *output vector* using the RNN by taking for every clause the representation

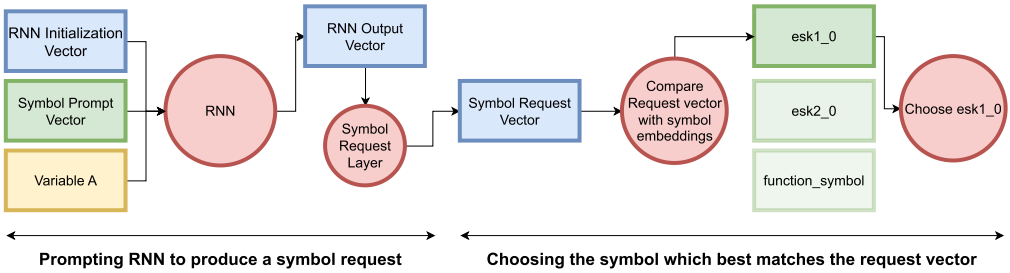


Fig. 5. RNN architecture, shown predicting a symbol for Variable A, the first variable in a clause. A special, trainable *symbol prompt vector* is used to mark a step where the RNN must predict a symbol for the queried variable. The vectors representing Variable A and the symbols *esk*1_0, *esk*2_0 and $function\_symbol$ are created by the GNN. GNN and RNN are trained end-to-end as one. Variable vectors are colored yellow and symbol-related vectors are green. Blue indicates that the vector is an intermediate vector or an RNN state. Red indicates a transformation, calculation or a choice.

of the first variable that occurs (in de Bruijn order) from the set **T**, a special *symbol prompt vector* and an initialization vector.

This RNN output vector is processed by the *symbol request layer* to give a *symbol request* vector. The symbol request layer is a neural network linear layer that has as its task to transform the RNN output vector into a vector similar to the vector representing the required symbol. We compute the dot product between this request vector and the representations of each function symbol in the signature. We then apply the softmax function to get a probability distribution over the function symbols for that variable. We then continue with the next step of the procedure (see Fig. 6), where the RNN gets its own output from the previous step, the chosen symbols, as well as the representation of the second variable.

2.5.3. Conditional prediction

After having chosen a symbol for the previous variables, we want to choose the next symbols while taking into account our earlier choices. To create symbol predictions that are conditioned on the symbols that were already chosen for other variables in the current clause, we created the setup as shown in Fig. 6. There, we schematically show the network in the process of predicting a symbol for the second variable (B) in a clause. First, the network gets as its input an initialization vector, a variable representation vector and the representation vector of the symbol that was already chosen (esk1_0) for that variable (A). The RNN then produces an output vector that should encode all the information about these prior decisions that is necessary to predict the next symbol. In the second step, the RNN processes this output vector, a vector representation of the second variable and a special *symbol prompt vector* that indicates that the loading of previous decisions has ended and that we are currently expecting a new symbol prediction. The RNN then produces a new output vector, which we

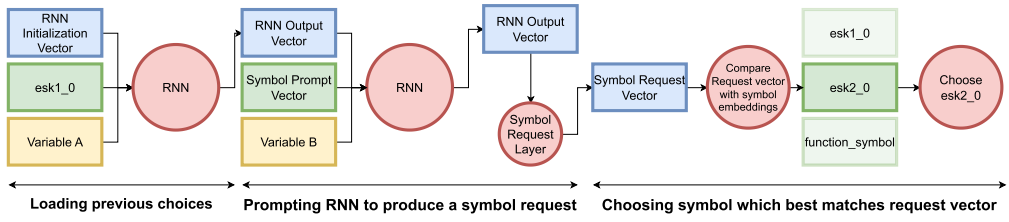


Fig. 6. RNN architecture, shown in the process of predicting a symbol for the second variable B in a clause. *esk*1_0 and *esk*2_0 are two symbol node representations (for Skolem constants). Here *esk*1_0 was chosen for Variable A. Before using the *symbol prompt vector*, the prefix of currently assigned symbols is shown to the RNN and the RNN can predict a symbol for variable B conditioned on the choice for A.

process as a symbol request using the symbol request layer. A symbol is then chosen according to the probability distribution induced by the dot products of the request and the symbol vectors, as before.

During training, we maximize the probability of choosing the symbol used in the known proof. During evaluation (Section 3) we can either (i) *decode greedily*, choosing the maximum probability symbol, (ii) *sample symbols* according to the distribution defined by the model (which introduces some randomness), or (iii) use a *beam search* procedure to find the most likely sequences. In this work, we always use the sampling method (ii), as it can produce varied output and is easier to implement than beam search.

This procedure preserves the anonymity of the entire setup: the system is invariant to naming. In the end, we obtain a mapping of variables to symbols for each clause. In addition, the network can choose a special *stop* symbol (represented by a special trainable vector) when shown the first variable, which indicates that the clause should not be instantiated. This means the RNN predictor is first performing a premise selection task and then an instantiation task when the clause is selected. After all variables are processed, we show the first variable again. The RNN can either produce stop, or another symbol to continue producing another instance for the clause. If the RNN predicts a stop symbol at a point only some variables have been processed, we do not use the sample. For our purposes here, we only produce instantiations where all variables in a clause have been replaced by a symbol (which possibly has new variables associated with it).

2.6. Dataset of mathematical problems

We use a dataset of 113 332 first-order ATP problems made available to us by the AI4REASON project.[1] They originate from the Mizar Mathematical Library (MML) (Kaliszyk and Urban, 2015) and are exported to first-order logic by the MPTP system (Urban, 2006). All these problems have an ATP proof (in general in a high time limit) found either by the E/ENIGMA (Schulz, 2013; Jakubův et al., 2020) systems or by the Vampire/Deepire (Kovács and Voronkov, 2013; Suda, 2021) systems. Additionally, the problems' premises have been *pseudo-minimized* (Kaliszyk and Urban, 2014) by iterated Vampire runs. In this procedure, input clauses are dismissed when they are not necessary for the proof. We use the pseudo-minimized versions because our focus here is on guiding instantiation rather than premise selection. The problems come from 38 108 *problem families*, where each problem family corresponds to one original Mizar theorem. Each theorem can have multiple minimized ATP proofs using different sets of premises. The problems range from easier to challenging ones, across mathematical fields such as topology, set theory, logic, algebra and linear algebra, real, complex and multivariate analysis, trigonometry, number and graph theory.

While we have 113 332 problems in the full dataset, a smaller subset was selected to allow quicker iteration for some of our experiments. The problems selected correspond to 2003 Mizar theorems known as the M2k subset, which is a subset of related Mizar articles (Kaliszyk et al., 2018). Since we typically have multiple premise selections proving the same Mizar theorem, 4817 problems constitute the dataset which we will refer to as our *M2k Dataset*.

2.7. CC + SAT implementation

CC + SAT is a standard procedure used in state-of-the-art SMT solvers such as CVC5 (Barbosa et al., 2022). After some experiments comparing several CC + SAT implementations, we have chosen the implementation provided in the Vampire system (Voronkov, 2014; Kovács and Voronkov, 2013). The choice for using Vampire as CC + SAT backend was made after also testing CVC5. We have found that Vampire's parser was faster, while there is no difference in the solved problems. We always use a 30 s time limit for the ground solver and a faster implementation allows a higher number of ground instances to be given to the CC + SAT system. For more information, see also Appendices A, B and H. Note that all clauses with variables are removed before calling Vampire in its CC + SAT mode.[2]

[1] https://github.com/ai4reason/ATP_Proofs.

[2] We call Vampire with the *acc* argument.

This means that Vampire is really used only as a standard CC + SAT solver for ground problems and no other parts of Vampire are being used for instantiation. In Section 2.4, we explained how our procedure can produce non-ground clauses at each level, which could potentially be made ground at the next level by instantiation with constants.

2.8. Generating training data via random grounding

To create our initial training dataset of instantiations, we use a randomized grounding procedure. We first clausify the problems using E (Schulz, 2013), and then repeatedly run a randomized grounding procedure on all of them, followed by the ground solver.

Randomized grounding can be parameterized in various ways. To develop the initial dataset here we use a simple multi-pass randomized grounding with settings that roughly correspond to our ML-guided instantiation architecture (Section 2.5). These settings are as follows. We use at most two passes (levels) of instantiation for every input clause. In the first pass, for each variable we randomly select an arbitrary function symbol from the problem's signature, and provide it (if non-constant) with fresh variables as arguments. In the second pass, we ground all variables with randomly selected constants from the problem's signature. The first pass is repeated 25 times for each clause in its input, and the second pass 5 times. The input to the first pass is the original clausal problem. The input to the second pass is the (deduplicated) union of the clauses produced in the first pass and of the original clauses.

This means that each input clause can produce up to $(25 + 1) \times 5 = 130$ ground instances, potentially resulting in ground problems with thousands of ground clauses. Such input sizes typically pose no problems to the ground solver. The average ground problem sizes are however typically below 1000, because of the overlaps and limited number of options during the random grounding.

The first run solves 3897 of the problems, growing to 7790 for the union of the first 9 runs, and to 11 675 for the union of the first 100 runs. The 113 332 problems have on average 35.6 input clauses and the 3897 problems solved in the first run have on average 12.8 input clauses. There are 6.0 instances needed on average to solve a problem.[3] Note that each clause can in general be instantiated more than once. Also, 3.9 (almost two-thirds) of the instances contain on average at least one Skolem symbol. For these symbols, the name is not necessarily informative. These data statistics confirm that we would strongly benefit from a learning architecture invariant under symbol renaming, rather than off-the-shelf architectures (e.g., transformers) that depend on fixed consistent naming. This motivates our use of a naming-invariant architecture (Section 2.5). Appendix C contains some more information on the random instantiation.

2.9. Neural network data processing and training details

In the following subsections, we will discuss practical details of the neural network training process, such as the data split for evaluation purposes, the preparation of the data, the balancing of the loss contributions from different samples and hyperparameters. Appendices E, F and G contain some more details on the neural network choices and the training.

2.9.1. Data split

For the machine learning experiments, the Mizar theorems were split into 90% training and validation data and 10% testing data. Of the largest set, 5% of data was used as the validation set and 95% as training data. Note that our split keeps problems which are versions of the same theorem in the same section of the split, so that data leakage between minor variations of the same proof idea is prevented. Also, **the fact that a problem is assigned to the training set does not imply we already have a proof from the randomized grounding: the split is done independently of the availability of a solution.** This means that each of these sets is a mixture of problems that already have a proof from the randomized grounding and problems that do not have one. In the end, we have 96 532 training,

[3] Over 40 instances (max. 63) are used in some problems.

5293 validation and 11 507 test problems. For the M2k subset, we have 4163, 188 and 466 problems in the respective sets.

2.9.2. Hardware

All experiments were run either on a DGX machine with 8 NVIDIA Tesla V100 GPUs with 32 GB memory, 512 GB RAM and 80 cores of Intel(R) Xeon(R) CPU E5-2698 v4 @ 2.20 GHz type (looping and running provers) or on a machine with 4 NVIDIA GTX 1080 GPUs with 12 GB memory, 692 GB RAM and 72 cores of Intel(R) Xeon(R) Gold 6140 CPU @ 2.30 GHz type (only training).

2.9.3. Data preparation

The initial training data were the cumulative proofs obtained by 9 runs of the random instantiator with 25 samples on level_0 and 5 on level_1. For the full dataset, there are 6592 solved training problems, while for the M2k set, there are 421. If multiple proofs were found for a problem, one of those proofs was chosen randomly. Note that the number of solved problems here can be lower than the number of available training problems. If a given proof needed two levels of instantiation, the proof is split into 2 training examples E_1, E_2. The input part of E_1 is the original CNF problem, while for E_2 it is the CNF problem with the right head symbols (and corresponding fresh variables) filled in for the proof-related clauses. For E_1, the label part corresponds to the head symbols in the input of E_2, while the labels for E_2 are the constants that ground the terms. The parameters of the neural network are trained by minimizing the cross-entropy between the predicted distribution and the labels.

When there are multiple instantiations for the same clause needed for the proof, we concatenate all the proof instantiations for a clause into a single sequence. The RNN component was trained to predict this concatenated sequence of instantiations so that the model can capture the conditional dependence between multiple instantiations of one clause. To make it possible for the model to stop instantiating a clause, a special *stop* vector is added in addition to the actual symbol vectors when comparing with the symbol request vectors. The label corresponding to the *stop* vector was added to the end of each label sequence. For clauses where no instantiation is part of the proof, the label sequence is only the *stop* label. Therefore premise selection is included in the task.

While we choose to handle multiple instances for a clause sequentially with an RNN, there is no ordering on these instances for a given clause. Therefore, during training, we randomize the order in which the different label sequences corresponding to different instances are concatenated. Each time a sample is encountered, we use a randomized order (so over the training process, many different orderings are observed, as a form of data augmentation). In principle, the same holds for the ordering on the variables, but this ordering is not randomized in the current setup, to limit the computational resources needed. The variable order is as they appear in the clause.

2.9.4. Loss balancing

The number of choice points, and thus the number of contributions to the total loss when naively added, is not the same for each training example. Some training examples require as many as 100 symbols to be chosen, while others need less than 10. Therefore, we normalize the loss contribution coming from each training example in the batch by the number of choice points (i.e. the total length of all the concatenated label sequences for all clauses with variables in the training example). We then minimize the sum of these averages. This loss corresponds better to our use case: it is more important to get all 3 instances for a small problem, than it is to get 3 out of 80 instances for another bigger problem.

2.9.5. Hyperparameters

The GNN was used with node and layer dimensions, for all nodes, set to 64. The network uses 10 message passing steps, with different layer parameters at each step. Each different node type is initialized with a different trainable initial embedding of 64 numbers. The RNN consists of a linear neural network layer with 192 inputs and 64 output dimensions followed by a rectified linear unit activation function, followed by a linear layer of input size 64 and output size of 64. The symbol request layer is a linear layer with 64 input and 64 output dimensions. To optimize the parameters of the network, the ADAM algorithm (Kingma and Ba, 2015) was used with *learning rate* 0.0001,

minimizing the cross entropy between the symbols used in the known proofs and the predicted symbols. The maximum number of RNN iterations per input clause (which limits the total number of symbols chosen per clause) was set to 12.

Parameter settings such as layer size and learning rate were determined by increasing them (decreasing in the case of learning rate) until the network reliably converged to a low loss on the training dataset, indicating a capacity to fit the data. In the end, we settled on 10 layers and 64-wide embedding dimensions, starting from smaller ones. While this seems enough to fit the data reasonably, we did not test networks that were deeper or wider than this. It is possible that more performance can be gained with more extensive examination of the various hyperparameters.

3. Results

To characterize the performance of our setup, we carried out several experiments. First, we explain the training of the initial network from the dataset created by randomized grounding and how we selected specific predictors. We also measure how well the predictors can generalize to samples not in the training data.

Second, we show the results of an iterated loop on the M2k dataset, where we take this trained predictor and use it to predict solutions, add the new solutions to the training data, and repeat. We use the same CC + SAT setup as with the randomized grounding procedure. After showing that this iterated procedure works, we also iterate on the full dataset, where we show how the system can self-improve up to the point of proving some theorems existing automated theorem provers did not find in 60 s.

3.1. Prediction model selection

For the looping and evaluation experiments, we use the models that have the earliest highest median validation accuracy (indicating that after this, the model could start overfitting). For the predictor trained on the M2k set, this happened after 56 passes over the training data. For the training on the full dataset, we use checkpoint 79, for the same reason.[4]

3.2. Validation and instance accuracy of the neural networks

The training setting does not fully reflect how the model is used for the real instantiation task. In the training procedure, accuracy is computed while the input and the previous choices are always fully correct (i.e. according to the labels). However, in the practical setting, the model can only run on its own previously generated, and possibly suboptimal, output. This is explored in Table 1.

The results are for the model trained on the data from the proofs generated by the random instantiator, but applied on unseen validation data and compared to unseen proofs from the random instantiator (which may have 2 levels of instantiation). We show the fraction of label instances covered as a function of the number of samples taken per clause (more simply, the accuracy). The accuracy values correspond to certain quantiles q. For example, a 0.67 score for $q = 0.1$ means that for 10% of all problems, at least 33% of the required instantiations are still missing. While the median ($q = 0.5$) and the 90th quantile indicate that with 25 samples, most instantiations are covered, the $q = 0.1$ data show that there is a subset of problems for which instances are missing. We also see that, given correct instantiations on level_0, filling in the right instances on level_1 (bottom 3 rows) is easier than starting from the base problem (top 3 rows). In general, there is an indication that the predictor has learned the task. Next, we show our iterative self-improvement experiments.

[4] In Appendix D, we show a plot of accuracy during the training process.

Table 1
Coverage of the instantiations needed for the proof, on problems from the validation set that the random instantiator found a proof for. The columns indicate the maximum number of instantiations per clause (which are sampled from the probability distribution learned by the model). The rows are split into the results for level_0 and level_1. The accuracy corresponding to the quantiles 0.1, 0.5 and 0.9 are given. Note that even if the instantiations of the proof in our validation set for a given problem are not covered, the predicted instances might constitute a different proof.

No. samples / clause	1 inst.	5 inst.	10 inst.	25 inst.
level_0 - q=0.1	0.08	0.40	0.53	0.67
level_0 - q=0.5	0.50	0.83	1.00	1.00
level_0 - q=0.9	1.00	1.00	1.00	1.00
level_1 - q=0.1	0.00	0.21	0.50	0.82
level_1 - q=0.5	1.00	1.00	1.00	1.00
level_1 - q=0.9	1.00	1.00	1.00	1.00

3.3. Self-improving loop (M2k dataset)

The previous section indicates that the model has learned to recreate the right instances for many proofs on unseen data. Next, we test whether the system can generate new proofs, and then learn from these new proofs, in a self-improving loop. To test this capability, we do a looping experiment. We use two levels of instantiation, limiting the maximum number of instantiations per clause to 25 and 5 respectively. This way we can compare to the random instantiator baseline, which also uses 2 levels of instantiations with the same limits.

We run the system on problems from the training set, including those where the random instantiator could not find a proof. We keep all the proofs, and train again including the new proofs. This procedure is then repeated.

Because one full loop iteration can take time, we first test this setup on the M2k subset. There are 4163 problems derived from the M2k theorems assigned to the training set. Of these, the random instantiator solved 421 total in 9 runs (this is the initial training data). In 100 runs (416 300 attempts total), it found 609. In each iteration, 1000 problems are attempted and 1000 random previous proofs are trained on (but the model parameters are kept between iterations). Every 10 iterations, we also run the proof attempts on the test set.

After 580 loop iterations, there are 1102 problems from the training set that we have a proof for (see Fig. 7). However, the discovery of new proofs stagnates after around 400 iterations. We concluded that restarting the training with a fresh model might bring more new proofs, as by this point the training solutions had been seen many times, which could lead to an overfit model. The restart is therefore a chance for the model to find a new optimum, that possibly better incorporates recently added solutions. As seen in Fig. 7, this restarts the process and the system finds 146 more proofs, for a total of 1248. The system found almost double the amount of proofs as the random instantiator (dashed red line) in the same amount of iterations, indicating that the learned distribution of terms can lead to more proofs.

In Figs. 8 and 9, the non-cumulative training and test set performance of the system are shown. The system learns how to prove around 22% of training problems, which corresponds roughly to the ratio between 1248 solved training problems and 4163 total training problems. Most training problems that were solved once can be solved reliably after training on their solutions. On the M2k test set, the behavior is similar, although the unseen data makes it harder: the system can prove 14.5%, which is more than double the performance of 1 run of the random instantiator with the same sampling settings (6.9%). The extra performance gain after the restart is noteworthy. This indicates that the optimization found a more generalizable optimum when trained from scratch with the solutions found before the restart of the self-improvement loop.

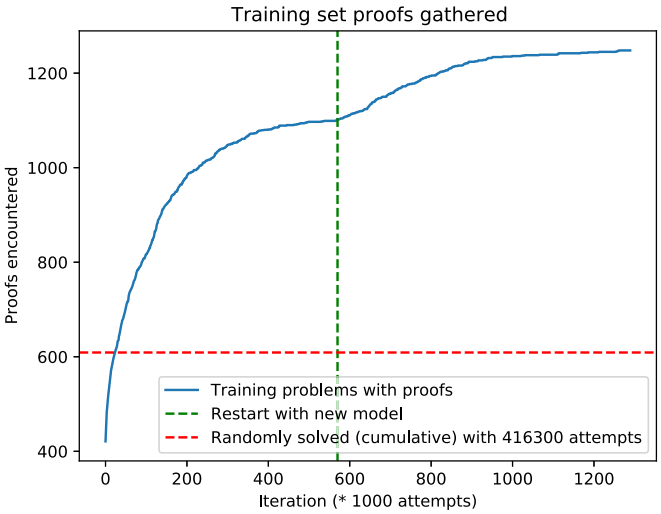


Fig. 7. Number of problems with a solution (cumulative, M2k). Horizontal dashed line indicates the number of problems solved by random attempts and vertical dashed line indicates the time at which a newly trained model was introduced.

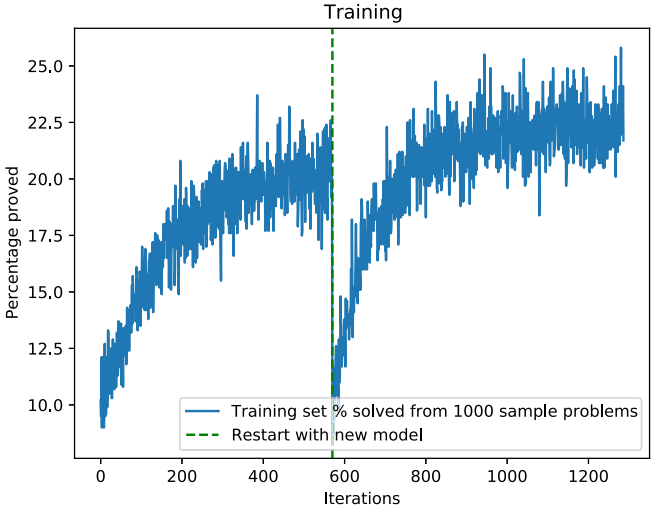


Fig. 8. Percentage of problems solved (training set problems, M2k).

3.4. Self-improving loop (full dataset)

As the self-improvement loop was successful on the M2k dataset, the experiment was repeated for the larger, full dataset. For this larger experiment, the system uses 5 levels of instantiation, with $(25, 5, 5, 5, 5)$ maximum samples per clause for the respective levels. There are 10 000 proof attempts and 10 000 training proofs are randomly selected from all proofs at each iteration.

We start from the 6592 training problems solved by the random instantiator. After 550 iterations, with a restart of the model included, there are 31 003 problems solved, more than $4.5\times$ the number of solved problems the procedure started with. This means that the system cumulatively found proofs

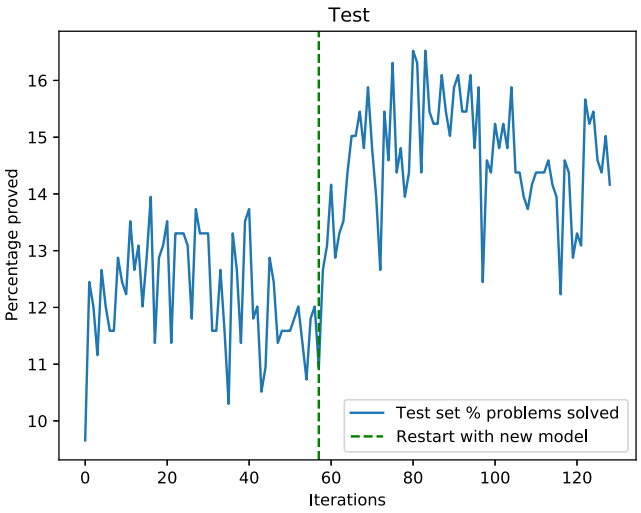


Fig. 9. Percentage of problems solved (test set problems, M2k).

of 32.12% of the training dataset. With the predictor from the last iteration, the system can prove 26.25% of training set problems. Most of the previously cumulatively solved training problems are solved by the final system. In Fig. 10, we can see the progression of test performance. Of the full set of unseen test problems, the final system can solve 19.74%.

The depth of the iteration procedure, or the number of levels, has an effect of the number of proofs found. In general, allowing more levels improves the performance. However, the number of clauses that serve as input for the next level potentially grows as fast as the limit on the number of samples per clause. Therefore, the smaller the number of levels, the faster the system runs in general. We tried depths from 2-5, but there were diminishing returns in terms of number of proofs and rising computational costs, which is why 5 was chosen as the limit.

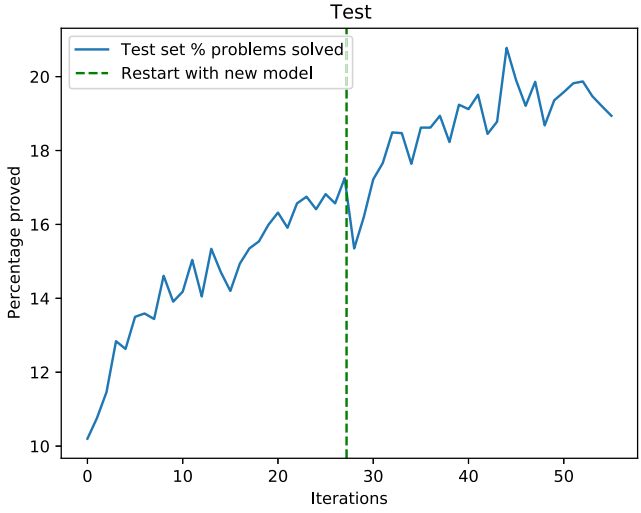


Fig. 10. Percentage of problems solved (test set problems, full set, sample of 10 000 problems taken per iteration).

3.5. Comparison with existing provers

Our system takes about 1 second per level of instantiation, plus a maximum of 30 s for the ground solver (though most ground problems are solved in under 1 second). We compare the performance of our system with existing automated provers. Table 2 shows the percentage of solved problems for various solvers and Table 3 shows how many problems our system solved that were not solved by the existing automated theorem provers. For each prover, there are some solutions that our system adds. For iProver in instantiation mode, the set of added solutions is the largest, but even for a highly optimized SMT solver like CVC5, we can add some solutions. With the final cumulative 31 003 solutions, we add 6821 new solutions compared to 1 s iProver on the training problems, or a 16.33% gain. On the test set (non-cumulative, last iteration predictor) we add 317, or a 6.38% gain.

Table 2
Performance of various methods. iProver is used in pure instantiation mode. Random is 1 run of the 2-level random grounding. In parentheses, we indicate which dataset was used.

Time limit	1 s	10 s	60 s	Inst. + 30 s
Random (all)	-	-	-	3.44%
Neural (train)	-	-	-	26.25%
Neural (test)	-	-	-	19.74%
iProver (train)	43.28%	59.99%	67.6%	-
iProver (test)	43.16%	59.75%	68.69%	-
CVC5 (test)	83.44%	85.6%	86.28%	-

If we compare to the longest iProver run, there are still 97 test set problems not found by iProver in 60 s. Some types of problems seem to be dominant: 15 of the new solutions are about asymptotes, while 9 are about quaternions and 7 are about complex numbers. While these types of problems are somewhat prevalent in the dataset, they are not the most prevalent. Problems about finite sequences, euclidean geometry and group theory are much more numerous.

Table 3
New solutions gained compared to existing provers. Solutions already contained in the random instantiator runs that serve as the starting training data are not counted.

Gain (# / %)	vs. 1 s	vs. 10 s	vs 60 s
vs iProver (train cumul.)	6821 / 16.33	3459 / 5.97	2471 / 3.79
vs iProver (test)	317 / 6.38	159 / 2.31	97 / 1.23
vs CVC5 (test)	75 / 0.78	73 / 0.74	73 / 0.74

4. Related work

In addition to the recent general work on synthesis of logical data mentioned in Section 1, there is recent work on choosing instantiations in automated reasoning using ML, but with a different focus than our work. Several examples are found in the SMT community, where gradient boosted tree algorithms were used to filter possible terms (Blanchette et al., 2019; El Ouraoui, 2021) and to rank them for the SMT solving procedure (Janota et al., 2022). These however work within the solving loop of an existing SMT solver, whereas we are synthesizing instances, attempting to do most of the non-ground reasoning within a trained neural network.

There is also work on synthesizing loop invariants, which is similar in spirit to what is attempted in this work (Si et al., 2018). A difference is that we are synthesizing many objects (instances of each

clause) simultaneously, whereas loop invariant synthesis is more concerned with a single object. Also, the specific grammar of loop invariants used is limited, but we jointly learn synthesis over arbitrary function signatures, within a single signature-invariant system.

5. Conclusion

We have developed a fully neural instantiation mechanism for many clauses at the same time. Starting from data generated by randomly instantiating variables in problems from a real-world mathematics dataset, the machine learning component can learn how to instantiate and improve based on its own newly found proofs. The final system, after self-improvement on the full dataset, can solve 26.25% of the training dataset, starting from a core of problems comprising 6.83% obtained by a random instantiator. The system can also solve 19.74% of the unseen test problems, indicating generalization capabilities. The total pool of solved problems is more than 4.5× larger after the self-improvement loop.

The combination of a neural instantiator with a strong ground solver with a congruence closure mechanism combines two techniques according to their respective strengths: the graph neural network that can learn global heuristics for the instantiation of first-order variables is combined with the fast and optimized reasoning components of SAT-based solvers to propagate the consequences of the instantiations.

While the current system does not outperform highly engineered existing systems, such as iProver or CVC5, in absolute terms, we do gain new proofs compared to them. This indicates that the system has learned how to synthesize instantiations that are hard to reach for these systems. Taking these results in combination with the demonstrated self-improvement capabilities of the architecture, we believe that this opens up a new research direction of theorem proving systems that incorporate learned neural synthesis, applicable not only in instantiation, but also in other settings such as tableaux and saturation-style proving.

A possible future direction of research could be to close the loop in Fig. 1B: the neural network's prediction of instantiations could benefit from explicit information about the SAT model. Another direction could be the direct integration of a similar neural instantiation method into an SMT solver.

CRediT authorship contribution statement

Jelle Piepenbrock: Writing – review & editing, Writing – original draft, Software, Investigation, Conceptualization. **Josef Urban:** Writing – review & editing, Writing – original draft, Supervision, Software, Data curation. **Konstantin Korovin:** Writing – review & editing, Writing – original draft, Software. **Miroslav Olšák:** Software. **Tom Heskes:** Writing – review & editing, Supervision. **Mikoláš Janota:** Writing – review & editing, Writing – original draft, Supervision, Software.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The article contains a link to a repository with code and data.

Acknowledgements

This work was partially supported by the European Regional Development Fund under the Czech project AI&Reasoning no. CZ.02.1.01/0.0/0.0/15_003/0000466 (JP, JU), the European Union under the project ROBOPROX (reg. no. CZ.02.01.01/00/22_008/0004590) (MJ), Amazon Research Awards (JP, JU), by the Czech Ministry of Education, Youth and Sports under the ERC CZ project POSTMAN no. LL1902 (JP, MJ, JU), EPSRC grant EP/V000497/1 UK (KK), and the EU ICT-48 2020 project TAILOR no. 952215 (JU).

Appendix A. Congruence closure + SAT

After the ML instantiator has instantiated the clauses in a problem, we remove all clauses that still have variables and send the remaining, ground part of the problem to Vampire. We use Vampire with the $--acc$ option that activates congruence closure and use a time limit of 30 s. However, the vast majority of executions finish in under 1 second.

Appendix B. Comparison with existing provers

We compare with iProver, another system that in one of its modes relies mainly on instantiation to prove problems. We use the following flags for iProver, to confine it to a pure instantiation-based mode:

```
--schedule none
--superposition_flag false
--resolution_flag false
--inst_prop_sim_given false
--inst_prop_sim_new false
```

For our comparisons with CVC5, we used the default settings, with the newest version as of March 2023.

Appendix C. Random instantiator

In Fig. C.11 we include a plot of the number of training set proofs cumulatively found by the random instantiator on the M2k subset of the training set. For the full dataset, the random instantiator cumulatively solves 9923 training problems after 100 passes with two instantiation levels.

We have also attempted to extract a training set of instantiation proofs from iProver, but encountered difficulties extracting training data. Exactly tracing all the steps as they happened, transforming them so that the ML system can replicate them, is difficult, especially because of differences in handling of equality. As we expected similar difficulties with other systems, we chose to start from random instantiations, in the exact same setting as our ML predictor.

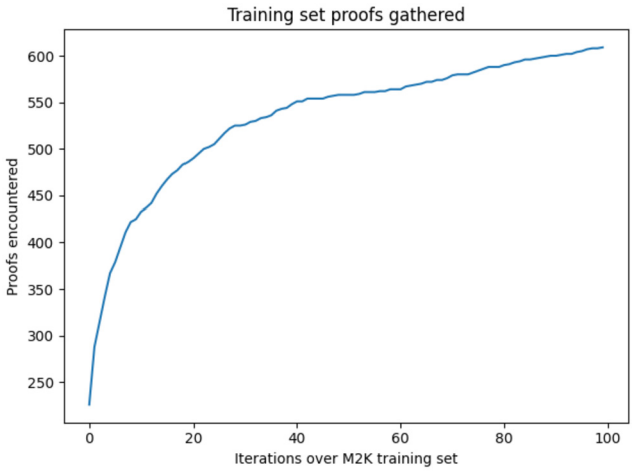


Fig. C.11. Cumulative M2k solutions by random instantiation limited to the training set.

Appendix D. Training curves

The Figs. D.12 and D.13 correspond to the model training on the full (non-M2k) training dataset. We can see that the model can reach very high performance on the training set (even achieving 0.95 median accuracy), but stops improving on validation after about 40 epochs.

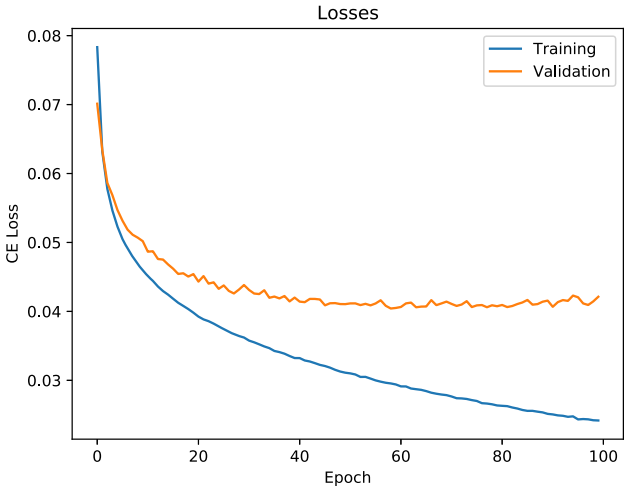


Fig. D.12. Loss curve for a model trained on the 25-5 random instantiator data.

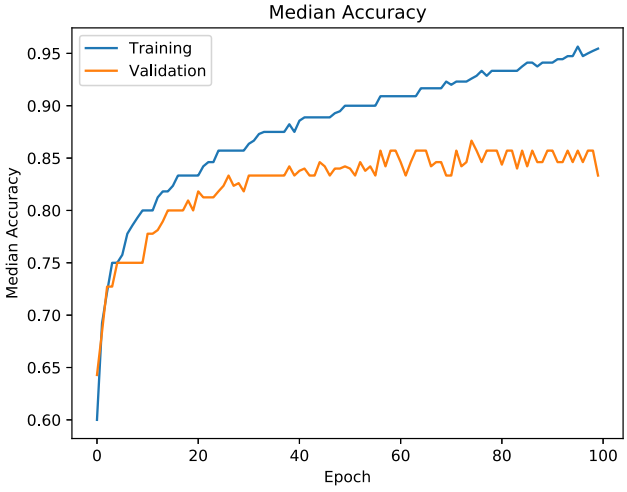


Fig. D.13. Median accuracy for model trained on the 25-5 random instantiator data.

Appendix E. Training on generated proofs

In some cases, during the looping experiments, multiple different proofs of the same problem can be generated. In that case, we keep the proof with fewer levels of instantiation (if there is a difference) to train on. The underlying idea is that we should train the instantiation system to be as concise as possible and that it should prefer proofs with shallower terms.

Appendix F. On keeping general clauses & training data alignment

When a clause is instantiated, we can either keep the parent clause of the instance around in our set of clauses, or the parent can be deleted. However, deletion would correspond to the assertion that no more instances of that parent clause are necessary anymore. As it is possible for the system to miss crucial instantiations, the parent clauses are kept so that they may be instantiated on the next level again. Deleting the parent clauses was also briefly tested, but led to worse preliminary performance.

This also leads to an interesting issue, as our training data is constructed to not include the parent clauses anymore, when the instances have been created (the training data is cleared of any superfluous steps). However, keeping the parent clauses in the training data did not lead to better results. This remains to be further investigated.

Another potential issue is that during inference time, the system produces unnecessary instantiations, which are then part of the input at the next instantiation level. This means that the input graphs start looking more and more dissimilar from the training data. Preliminary attempts to remedy this difference by also training on the data as it is during inference time however did not lead to better results. There may be a balance to strike in the relative importance of training on the various types of data.

Appendix G. Scaling & decomposition of the task

The GNN part of the approach scales at a maximum quadratically with the number of nodes (the sum of the number of clauses, literals and terms in the parsed problem), in the case of a complete graph. However, our graphs are not close to complete graphs. For the RNN part, the memory requirements scale linearly according to the number of clauses with variables in the problem C and also linearly with the number of required samples per clause (i.e. 25 or 5). The RNN time requirements in theory scale linearly with the size of the largest instance needed, measured in the number of variables. However in practice we cap the RNN at 12 iterations. The scaling of the method as a whole is dependent on the size (in terms of levels needed to generate the needed terms) of typical proofs and on the accuracy of the instances generated. If the probability distribution that is learned is very focused, only few unnecessary instances are generated, which keeps the growth of the clause set under control.

The looping experiments, where the network learns from a starting dataset to prove more and more theorems, can take days to complete. On the M2k dataset, one looping iteration (including training procedures) takes around 5 minutes (with 1000 attempts per iteration), whereas it takes around 50–60 minutes for the experiments on the full dataset (with 10 000 attempts per iteration). The M2k looping experiment was run with softmax temperature 2, and for the full dataset experiment the temperature was set to 1. The higher the temperature, the more unique instances are created. With 5 levels of instantiation, too many instances can slow down the procedure.

The total space of possible instantiations is impossible for us to examine. We have therefore chosen to use our level-based approach, where we construct the necessary terms from top to bottom. Other decompositions of the task are possible. For example, new terms could be constructed bottom-up, starting with already existing ground terms.

Having chosen to use the level-based approach, there are still other choices to make. For example, does the current state of the variable-symbol mapping in *other clauses* influence our choice of a symbol for a particular variable in our *current clause*? When considering the task itself, of course the instantiations of other clauses impact the viability of instantiations in the current clause. However, we have chosen to decompose the process in two parts: the graph neural network (GNN) can see the entire graph, but the recurrent neural network (RNN) that produces the instances can only see the variable embeddings for 1 particular clause (and to get instances for all clauses, we run the same RNN in parallel). For the system to work, the coordination of which instantiation strategy to pursue in the RNN must be happening in the GNN part, when the clause, symbol and variable embedding vectors are determined. We found this to be an acceptable trade-off, as it allowed us to consider each clause somewhat separate from the other ones, simplifying and speeding up our instantiation sampling pro-

cedure. A probability distribution that considers the combinatorial number of choices made in all the other clauses at all times would be very expensive to evaluate.

Appendix H. Using other ground solvers

We tested whether using CVC5 as our ground solver would change the results. We used three levels of neural instantiations on the test set problems, and removed all clauses with variables. Then, we ran either Vampire (with *acc* argument) or CVC5 (default parameters). We found no difference, with both systems solving the exact same amount of problems: 904, 1909, 2146 at the respective levels. This indicates that the precise choice of ground solver, as long as CC + SAT is present, does not influence our results.

References

Barbosa, Haniel, Barrett, Clark, Brain, Martin, Kremer, Gereon, Lachnitt, Hanna, Mann, Makai, Mohamed, Abdalrhman, Mohamed, Mudathir, Niemetz, Aina, Nötzli, Andres, et al., 2022. cvc5: a versatile and industrial-strength smt solver. In: International Conference on Tools and Algorithms for the Construction and Analysis of Systems. Springer, pp. 415–442.

Barrett, Clark W., Sebastiani, Roberto, Seshia, Sanjit A., Tinelli, Cesare, 2021. Satisfiability modulo theories. In: Biere, Armin, Heule, Marijn, van Maaren, Hans, Walsh, Toby (Eds.), Handbook of Satisfiability, second edition. In: Frontiers in Artificial Intelligence and Applications, vol. 336. IOS Press, pp. 1267–1329.

Blanchette, Jasmin Christian, El Ouraoui, Daniel, Fontaine, Pascal, Kaliszyk, Cezary, 2019. Machine learning for instance selection in SMT solving. In: AITP 2019 - 4th Conference on Artificial Intelligence and Theorem Proving. Obergurgl, Austria.

Chvalovský, Karel, Jakubův, Jan, Olšák, Miroslav, Urban, Josef, 2021. Learning theorem proving components. In: Das, Anupam, Negri, Sara (Eds.), Automated Reasoning with Analytic Tableaux and Related Methods - 30th International Conference, TABLEAUX 2021, Proceedings. Birmingham, UK, September 6–9, 2021. In: Lecture Notes in Computer Science, vol. 12842. Springer, pp. 266–278.

Clarke, Edmund M., Henzinger, Thomas A., Veith, Helmut, Bloem, Roderick, et al., 2018. Handbook of Model Checking, vol. 10. Springer.

Davis, Martin, Putnam, Hilary, 1960. A computing procedure for quantification theory. J. ACM 7 (1), 201–215.

Davis, Martin, 2001. The early history of automated deduction. In: Handbook of Automated Reasoning. Elsevier and MIT Press, pp. 3–15.

De Moura, Leonardo, Bjørner, Nikolaj, 2007. Efficient e-matching for smt solvers. In: Automated Deduction–CADE-21: 21st International Conference on Automated Deduction Bremen, Proceedings 21. Germany, July 17–20, 2007. Springer, pp. 183–198.

Detlefs, David, Nelson, Greg, Saxe, James B., 2005. Simplify: a theorem prover for program checking. J. ACM 52 (3), 365–473.

El Ouraoui, Daniel, 2021. Méthodes pour le raisonnement d'ordre supérieur dans SMT, Chapter 5, PhD thesis. Université de Lorraine.

Gauthier, Thibault, 2020. Deep reinforcement learning for synthesizing functions in higher-order logic. In: LPAR. In: EPiC Series in Computing, vol. 73. EasyChair, pp. 230–248.

Ge, Yeting, de Moura, Leonardo Mendonça , 2009. Complete instantiation for quantified formulas in satisfiability modulo theories. In: Computer Aided Verification, 21st International Conference, CAV, pp. 306–320.

Herbrand, Jacques, 1930. Recherches sur la théorie de la démonstration. Doctorat d'état La Faculté des Sciences de Paris.

Jakubův, Jan, Chvalovský, Karel, Olsák, Miroslav, Piotrowski, Bartosz, Suda, Martin, Urban, Josef, 2020. ENIGMA anonymous: symbol-independent inference guiding machine (system description). In: Peltier, Nicolas, Sofronie-Stokkermans, Viorica (Eds.), Automated Reasoning - 10th International Joint Conference, IJCAR 2020, Proceedings, Part II. Paris, France, July 1–4, 2020. In: Lecture Notes in Computer Science, vol. 12167. Springer, pp. 448–463.

Janota, Mikolás, Piepenbrock, Jelle, Piotrowski, Bartosz, 2022. Towards learning quantifier instantiation in SMT. In: Meel, Kuldeep S., Strichman, Ofer (Eds.), 25th International Conference on Theory and Applications of Satisfiability Testing, SAT 2022. August 2–5, 2022, Haifa, Israel. In: LIPIcs, vol. 236. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 7.

Kaliszyk, Cezary, Urban, Josef, 2014. Learning-assisted automated reasoning with Flyspeck. J. Autom. Reason. 53 (2), 173–213.

Kaliszyk, Cezary, Urban, Josef, 2015. MizAR 40 for Mizar 40. J. Autom. Reason. 55 (3), 245–256.

Kaliszyk, Cezary, Urban, Josef, Michalewski, Henryk, Olšák, Miroslav, 2018. Reinforcement learning of theorem proving. In: Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018. 3–8 December 2018, Montréal, Canada, pp. 8836–8847.

Kingma, Diederik P., Ba, Jimmy, 2015. Adam: a method for stochastic optimization. In: Bengio, Yoshua, LeCun, Yann (Eds.), 3rd International Conference on Learning Representations, ICLR 2015, Conference Track Proceedings. San Diego, CA, USA, May 7–9, 2015.

Korovin, Konstantin, 2008. iProver - an instantiation-based theorem prover for first-order logic (system description). In: Armando, Alessandro, Baumgartner, Peter, Dowek, Gilles (Eds.), Automated Reasoning, 4th International Joint Conference, IJCAR 2008, Proceedings. Sydney, Australia, August 12–15, 2008. In: Lecture Notes in Computer Science, vol. 5195. Springer, pp. 292–298.

Korovin, Konstantin, 2013. Inst-gen–a modular approach to instantiation-based automated reasoning. In: Programming Logics: Essays in Memory of Harald Ganzinger, pp. 239–270.

Kovács, Laura, Voronkov, Andrei, 2013. First-order theorem proving and Vampire. In: Sharygina, Natasha, Veith, Helmut (Eds.), CAV. In: LNCS, vol. 8044. Springer, pp. 1–35.

Nieuwenhuis, Robert, Oliveras, Albert, 2007. Fast congruence closure and extensions. Inf. Comput. 205 (4), 557–580.

Olšák, Miroslav, Kaliszyk, Cezary, Urban, Josef, 2020. Property invariant embedding for automated reasoning. In: De Giacomo, Giuseppe, Catalá, Alejandro, Dilkina, Bistra, Milano, Michela, Barro, Senén, Bugarín, Alberto, Lang, Jérôme (Eds.), ECAI 2020 - 24th European Conference on Artificial Intelligence, Including 10th Conference on Prestigious Applications of Artificial Intelligence (PAIS 2020). 29 August–8 September 2020, Santiago de Compostela, Spain. In: Frontiers in Artificial Intelligence and Applications, vol. 325. IOS Press, pp. 1395–1402.

Paszke, Adam, Gross, Sam, Massa, Francisco, Lerer, Adam, Bradbury, James, Chanan, Gregory, Killeen, Trevor, Lin, Zeming, Gimelshein, Natalia, Antiga, Luca, Desmaison, Alban, Kopf, Andreas, Yang, Edward, DeVito, Zachary, Raison, Martin, Tejani, Alykhan, Chilamkurthy, Sasank, Steiner, Benoit, Fang, Lu, Bai, Junjie, Soumith, Chintala, 2019. Pytorch: an imperative style, high-performance deep learning library. In: Wallach, H., Larochelle, H., Beygelzimer, A., d' Alché-Buc, F., Fox, E., Garnett, R. (Eds.), Advances in Neural Information Processing Systems, vol. 32. Curran Associates, Inc., pp. 8024–8035.

Schulz, Stephan, 2002. A comparison of different techniques for grounding near-propositional CNF formulae. In: Haller, S., Simmons, G. (Eds.), Proc. of the 15th FLAIRS. Pensacola. AAAI Press, pp. 72–76.

Schulz, Stephan, 2013. System description: E 1.8. In: McMillan, Kenneth L., Middeldorp, Aart, Voronkov, Andrei (Eds.), LPAR. In: LNCS, vol. 8312. Springer, pp. 735–743.

Si, Xujie, Dai, Hanjun, Raghothaman, Mukund, Naik, Mayur, Song, Le, 2018. Learning loop invariants for program verification. Adv. Neural Inf. Process. Syst. 31.

Silva, João P. Marques, Lynce, Inês, Malik, Sharad, 2009. Conflict-driven clause learning SAT solvers. In: Biere, Armin, Heule, Marijn, van Maaren, Hans, Walsh, Toby (Eds.), Frontiers in Artificial Intelligence and Applications. In: Handbook of Satisfiability, vol. 185. IOS Press, pp. 131–153.

Suda, Martin, 2021. Improving enigma-style clause selection while learning from history. In: CADE. In: Lecture Notes in Computer Science, vol. 12699. Springer, pp. 543–561.

Urban, Josef, Jakubův, Jan, 2020. First neural conjecturing datasets and experiments. In: CICM. In: Lecture Notes in Computer Science, vol. 12236. Springer, pp. 315–323.

Urban, Josef, 2006. MPTP 0.2: design, implementation, and initial experiments. J. Autom. Reason. 37 (1–2), 21–43.

Voronkov, Andrei, 2014. AVATAR: the architecture for first-order theorem provers. In: Biere, Armin, Bloem, Roderick (Eds.), Computer Aided Verification - 26th International Conference, CAV 2014, Held as Part of the Vienna Summer of Logic, VSL 2014, Proceedings. Vienna, Austria, July 18–22, 2014. In: Lecture Notes in Computer Science, vol. 8559. Springer, pp. 696–710.